Diagramas de domínio

Tech Challenge SOAT, fase 1, grupo 31

Domain Storytelling feito pelo grupo no egon.io (25/09/2026) e diagramas complementares de DDD (agregados, entidades, objetos de valor, contextos delimitados e máquinas de estado) montados a partir do board de Event Storming no Miro e conferidos com o código.

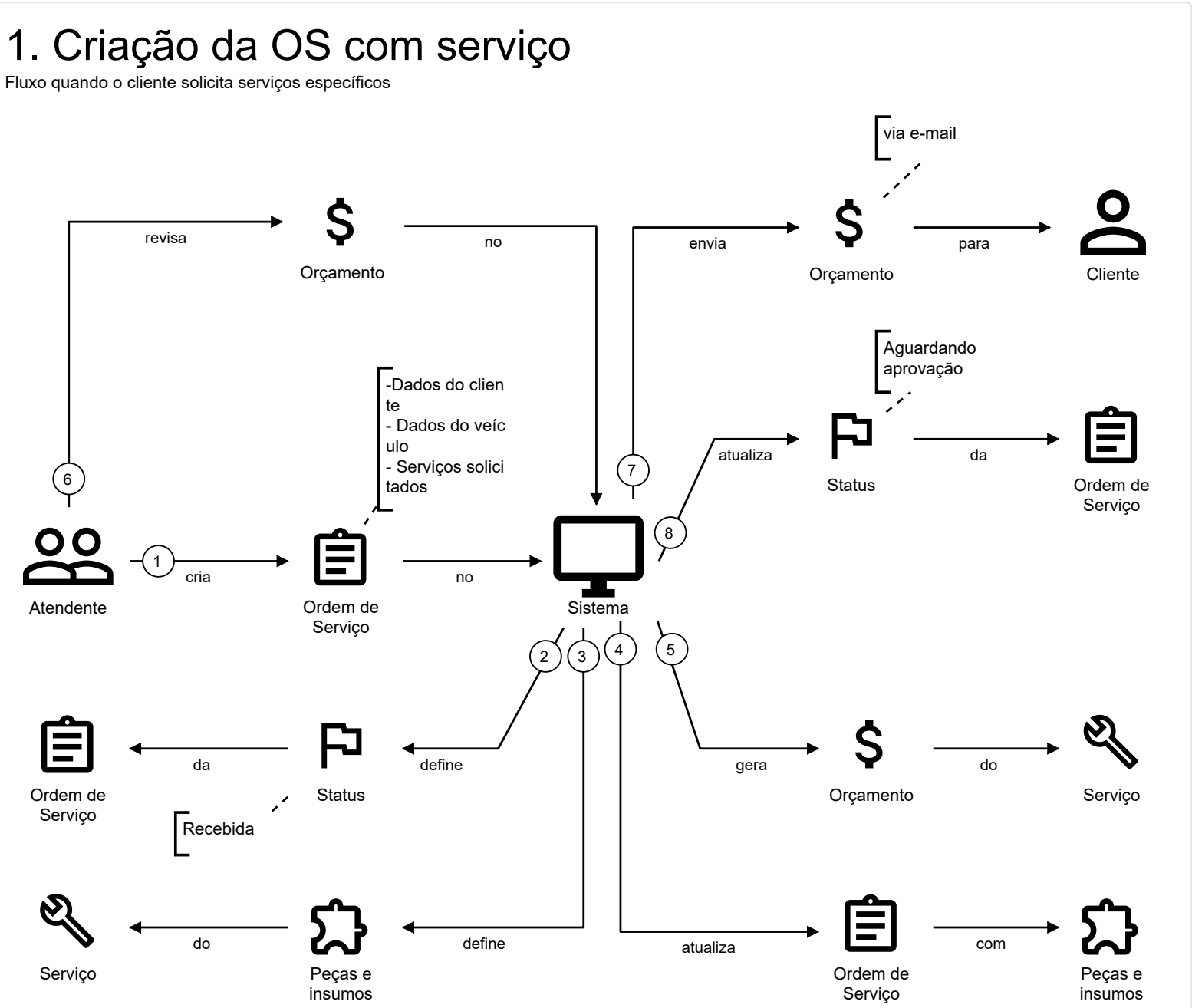
Conteúdo

Página	Diagrama	Origem
2	1. Criação da OS com serviço	egon.io (`docs/ddd/egon/1-criacao-os-com-servico.egn.svg`)
3	2. Criação da OS com diagnóstico	egon.io
4	3. Acompanhamento com aprovação total	egon.io
5	4. Acompanhamento com aprovação parcial	egon.io
6	5. Acompanhamento com reprovação	egon.io
7	6. Gerenciamento de peças e insumos	egon.io
8	Agregados, entidades e objetos de valor	Miro (agregados) + código
9	Contextos delimitados e mapa de contextos	Miro + código
10	Máquinas de estado da OS e do orçamento	código

Do lado de cada história colocamos como ficou no código. As diferenças também estão em `docs/ddd/linguagem-ubiqua.md`. O board do Miro: https://miro.com/app/board/uXjVHnerRBQ=/

1. Criação da OS com serviço

Fluxo quando o cliente já sabe quais serviços quer (ex.: troca de óleo).



História

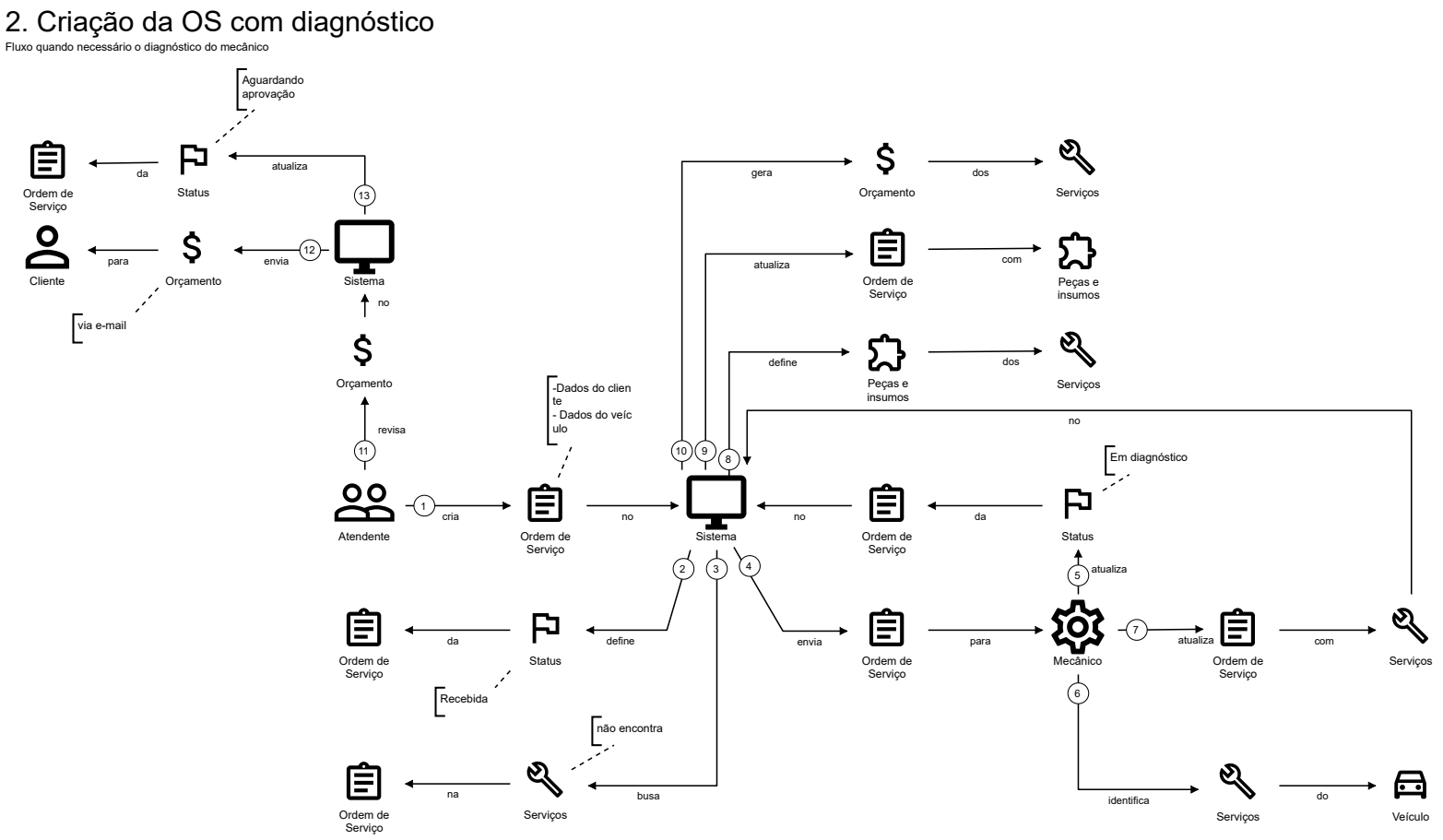
1. O atendente cria a OS com os dados do cliente, do veículo e os serviços pedidos.
2. O sistema define o status Recebida.
3. O atendente define serviços, peças e insumos; o sistema gera o orçamento.
4. O atendente revisa e o sistema envia o orçamento por e-mail; status Aguardando aprovação.

No código

- `POST /api/ordens-servico` abre a OS em `RECEBIDA`. O cliente é informado por `clienteId` **ou** `documentoCliente` (CPF/CNPJ, com ou sem pontuação) e o veículo por `veiculoId` **ou** `placaVeiculo`, um de cada par (400 se vierem os dois ou nenhum). Documento ou placa sem cadastro: 404; veículo de outro cliente: 422. Isso corresponde à nota do Miro "aqui ele busca por cpf/cnpj ou placa do veículo" (no board, junto à lista de ordens de serviço).
- Publica `OrdemServicoCriada`; o cliente recebe o código de acesso por e-mail (simulado em log).
- `POST /api/orcamentos` recebe os ids dos serviços do catálogo (nome e valor vêm de `ServicoRepository`) e as peças do estoque, reserva as peças e calcula o total. A OS passa por `EM_DIAGNOSTICO` e vai para `AGUARDANDO_APROVACAO` na mesma operação.
- Não existe etapa separada de "revisar"; o orçamento é enviado ao ser gerado (`OrcamentoGerado`).

2. Criação da OS com diagnóstico

Fluxo quando o problema precisa ser identificado antes do orçamento.



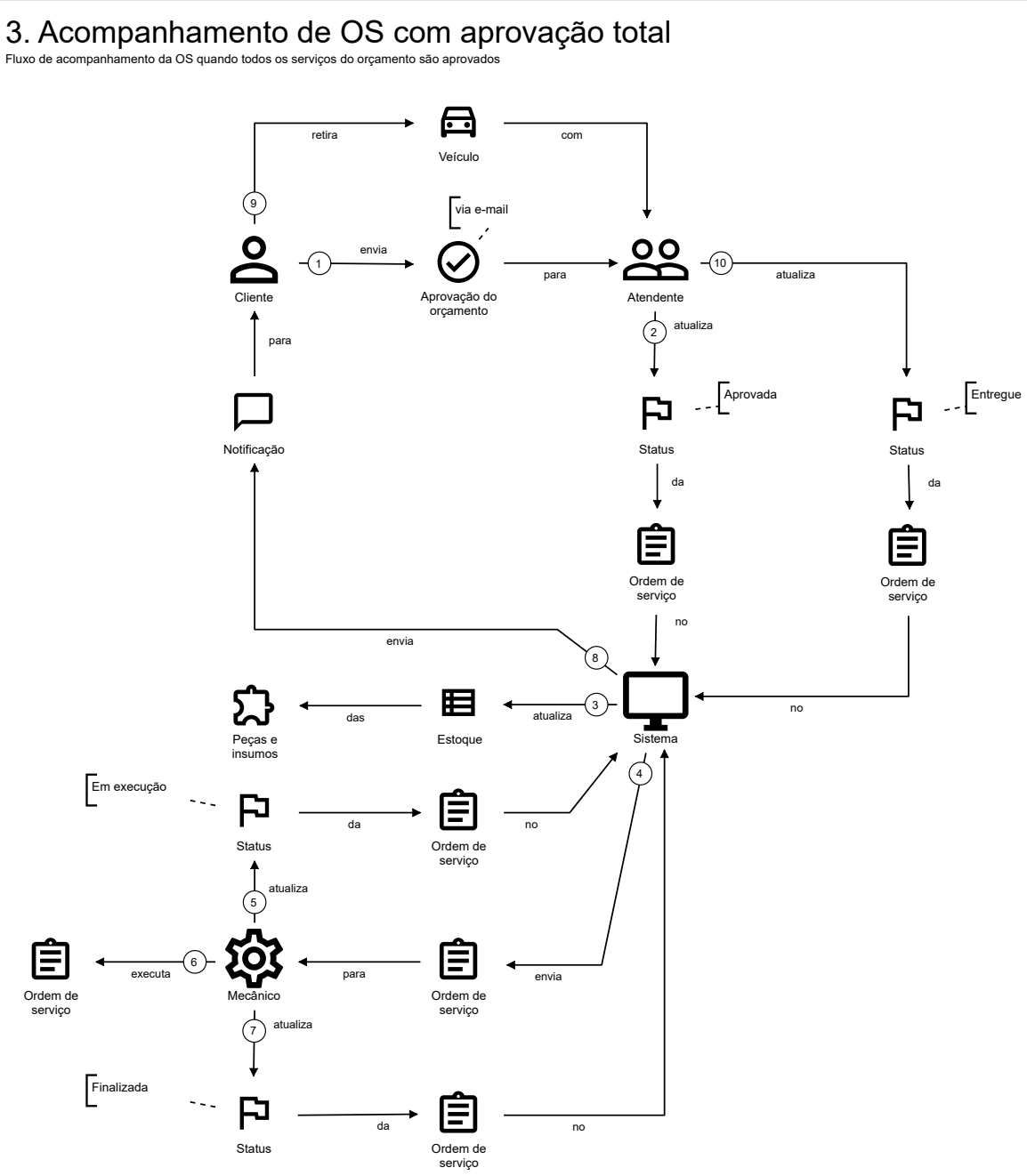
História

1. O atendente cria a OS com cliente e veículo; o sistema não encontra serviços e define Recebida.
2. A OS segue para o mecânico, que identifica os serviços (Em diagnóstico).
3. Os serviços, peças e insumos entram na OS e o sistema gera o orçamento.
4. O atendente revisa e envia por e-mail; status Aguardando aprovação.

> **No código**
>
> - O mecânico não opera o sistema (papel removido na migration V6). Ele faz o diagnóstico fora do sistema e o **atendente** registra o resultado ao gerar o orçamento.
> - O status `EM_DIAGNOSTICO` é marcado automaticamente pelo `GerarOrcamentoUseCase` quando a OS ainda está `RECEBIDA`.
> - Papéis do sistema: `ADMIN` e `ATENDENTE` (`Papel`).

3. Acompanhamento de OS com aprovação total

Todos os serviços e peças do orçamento são aprovados.



História

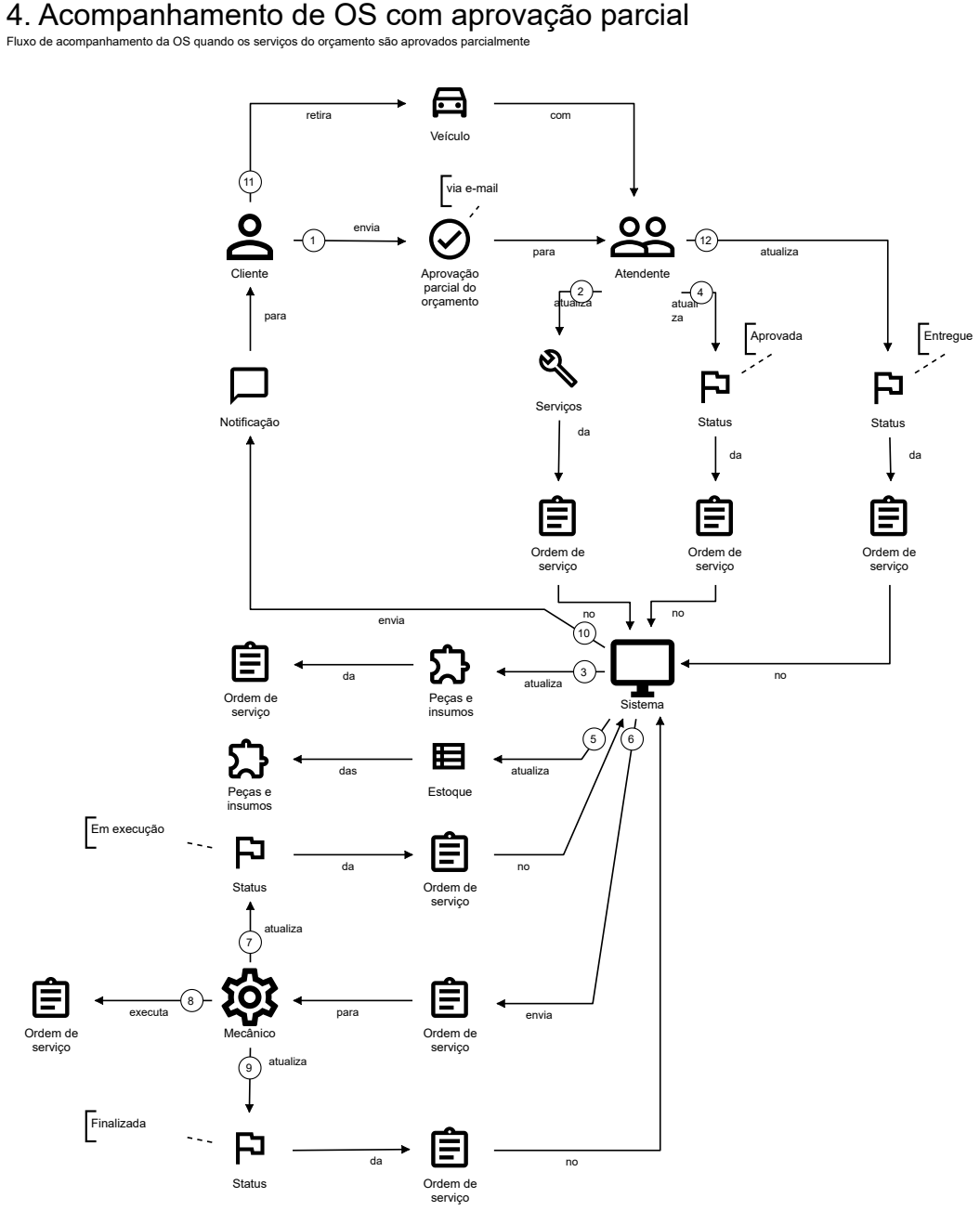
1. O cliente envia a aprovação do orçamento; status Aprovada e baixa de peças no estoque.
2. O mecânico executa (Em execução) e finaliza (Finalizada); o cliente é notificado.
3. O cliente retira o veículo; status Entregue.

No código

- Aprovação pelo cliente: `POST /api/publico/orcamentos/{id}/aprovar` com CPF/CNPJ e código de acesso; ou pelo atendente: `PATCH /api/orcamentos/{id}/aprovar`.
- Orçamento `APROVADO`, OS `APROVADA`, reservas viram baixa (`confirmarBaixa`), evento `OrdemServicoAprovada` com e-mail simulado.
- Execução e entrega são registradas pelo atendente: `PATCH /api/ordens-servico/{id}/iniciar-execucao`, `/finalizar` (e-mail "veículo pronto") e `/entregar`.

4. Acompanhamento de OS com aprovação parcial

O cliente aprova só parte dos serviços e peças.



História

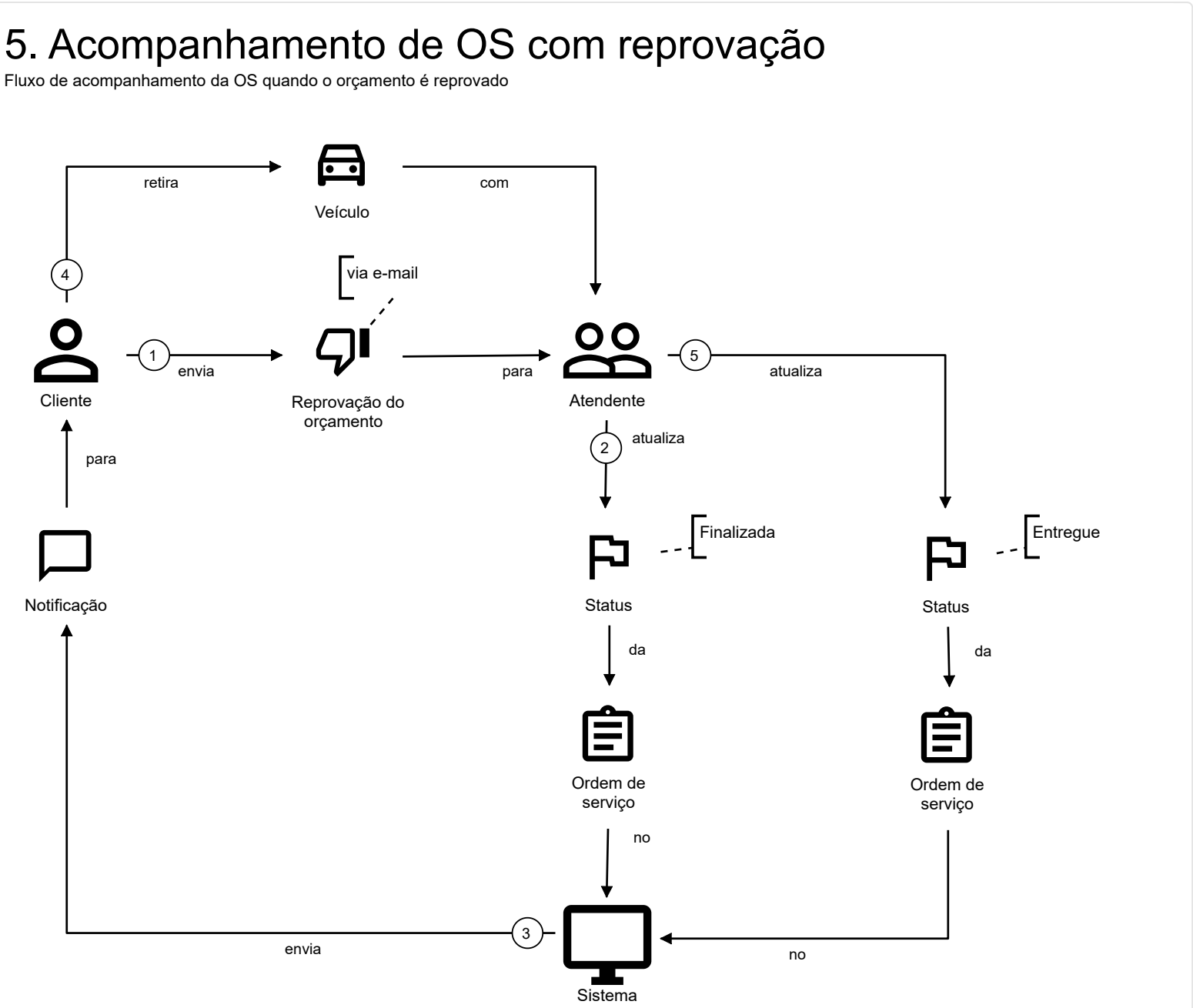
1. O cliente envia a aprovação parcial; a OS é atualizada só com os itens aprovados.
2. Baixa no estoque apenas das peças aprovadas; status Aprovada.
3. Execução, finalização com notificação e entrega como no fluxo 3.

No código

- Mesmo endpoint de aprovação, informando `servicosAprovados` e `pecasAprovadas`.
- `Orcamento.aprovar` remove os itens não aprovados, recalcula o total e marca `APROVADO_PARCIALMENTE`.
- Peças recusadas têm a reserva devolvida (`cancelarReserva`); as aprovadas recebem baixa.
- Se nenhum item for aprovado, o orçamento fica `REJEITADO` e a OS `RECUSADA`.

5. Acompanhamento de OS com reprovação

O cliente reprova o orçamento.



História

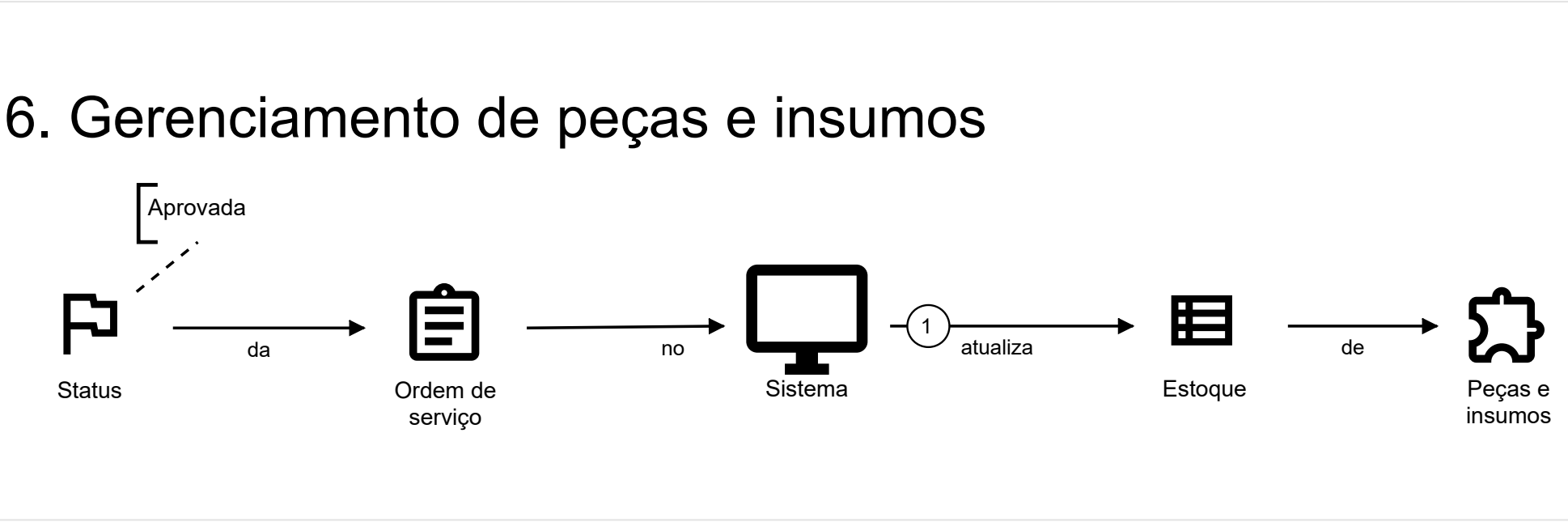
1. O cliente envia a reprovação; status Finalizada e notificação por e-mail.
2. O cliente retira o veículo; status Entregue.

No código (divergência)

- Reprovação: `POST /api/publico/orcamentos/{id}/rejeitar` (cliente) ou `PATCH /api/orcamentos/{id}/rejeitar` (atendente).
- O orçamento fica `REJEITADO`, as reservas voltam ao estoque e a OS vai para `RECUSADA`, que é **estado final**: não passa por Finalizada nem Entregue. O cliente recebe o e-mail "orçamento não aprovado".
- Assim, OS recusadas não entram no tempo de execução. A retirada do veículo, nesse caso, não é registrada no sistema.

6. Gerenciamento de peças e insumos

Na história do grupo: com a OS Aprovada, o sistema atualiza o estoque de peças e insumos.



Estoque dentro do fluxo da OS

- Orçamento gerado: `ProdutoEstoque.reservar` tira do disponível e soma no reservado (falha se faltar saldo).
- Orçamento aprovado: `confirmarBaixa` das peças aprovadas.
- Rejeição, aprovação parcial, expiração ou cancelamento da OS: `cancelarReserva` devolve ao disponível.

Gestão administrativa (`/api/estoque`)

- Cadastro e consulta de peças e insumos, com saldo disponível e reservado.
- Listagem, atualização e exclusão de peças ainda não foram implementadas.
- Entrada e saída manual de quantidade, que mexem só no disponível; reservas ficam com o orçamento.
- Nome da peça é único (restrição no banco).

Agregados, entidades e objetos de valor

No Miro o grupo separou os agregados Cadastro, Relatório, Orçamento, Estoque e Ordem de Serviço. No código, Cadastro virou dois agregados (`Cliente` e `Veiculo`), o Relatório foi absorvido pelo Orçamento (o diagnóstico vira os itens) e o catálogo de serviços virou o agregado `Servico`.

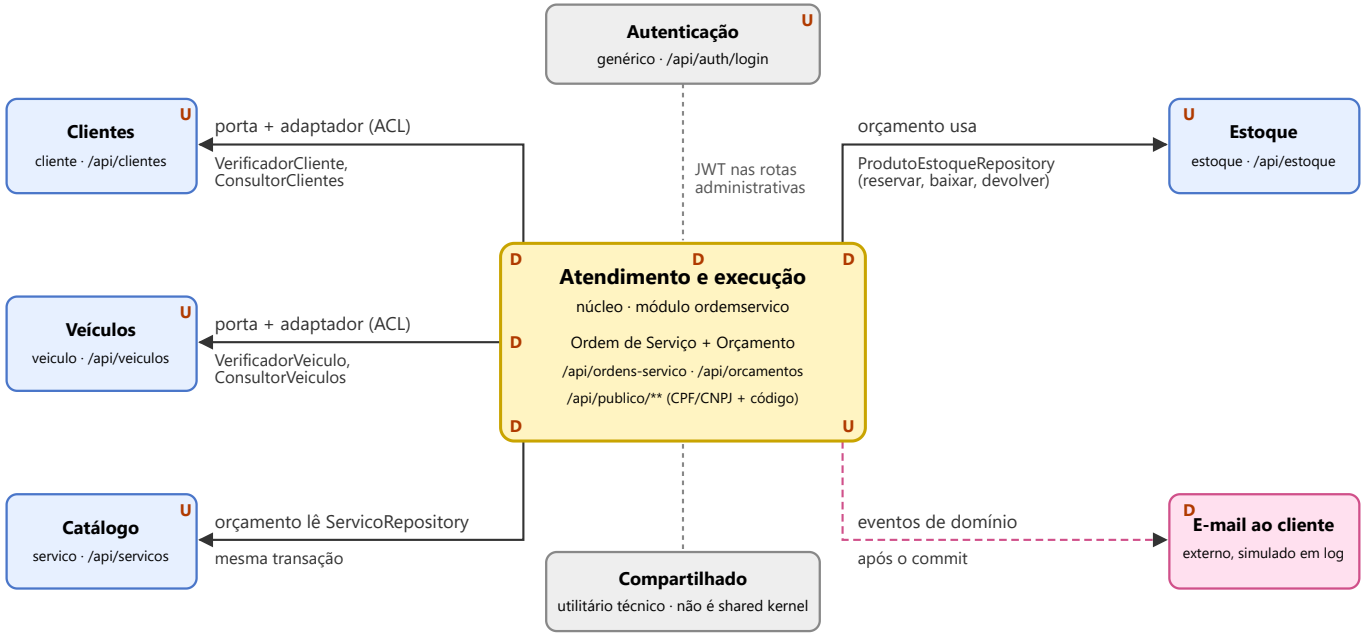
Agregado	**Raiz (entidade)**	**Outras entidades** (têm identidade própria)	**Objetos de valor** (definidos só pelos valores)	**Regras principais**
Ordem de Serviço	`OrdemServico`	`TransicaoStatus` (id, status anterior, novo e horário; tabela `historico_status_ordem_servico`)	`StatusOrdemServico` (enum); código de acesso	transições só pelas operações do agregado; cancelamento só antes da aprovação; relato de 10 a 500 caracteres. Eventos: `OrdemServicoCriada`, `OrdemServicoAprovada`, `OrdemServicoRecusada`, `OrdemServicoCancelada`, `OrdemServicoFinalizada`
Orçamento	`Orcamento`	-	`ItemServico` (serviço do catálogo, nome e valor da mão de obra copiados), `ItemPeca` (peça, quantidade, valor unitário), `StatusOrcamento` (enum)	só muda enquanto `PENDENTE`; total = mão de obra + peças; validade no futuro; aprovação total, parcial ou rejeição; aprovar depois da validade expira. Evento: `OrcamentoGerado`
Estoque	`ProdutoEstoque`	-	valor unitário, quantidades disponível e reservada	nome único; reservar só com saldo; baixa e devolução só sobre o reservado; entrada e saída manual só no disponível
Cliente	`Cliente`	-	`Endereco`, `Estado` (UF, enum); documento CPF/CNPJ	CPF/CNPJ com dígitos verificadores e único; não excluir com veículos
Veículo	`Veiculo`	-	placa (antiga ou Mercosul)	placa válida e única; não excluir com OS vinculada
Catálogo	`Servico`	-	valor da mão de obra (até 2 casas)	nome único; valor maior que zero; não excluir serviço já usado em orçamento
Autenticação	`Usuario`	-	`Papel` (enum: `ADMIN`, `ATENDENTE`)	senha guardada com BCrypt

Cliente e veículo são referenciados por id; a OS guarda os orçamentos dela. Itens de orçamento são objetos de valor (records Java) porque copiam o preço do momento e não mudam depois.

Fonte: Miro (agregados do Event Storming) e código (pacotes `*/domain`)

Contextos delimitados e mapa de contextos

Cada contexto é um módulo do monólito (`br.com.fiap.oficina_mecanica.*`), dividido em `domain`, `application` e `infrastructure`.



Legenda
amarelo núcleo · azul suporte · cinza genérico / utilitário técnico · rosa externo
U upstream (fornecedor) · **D** downstream (cliente, consome o U)

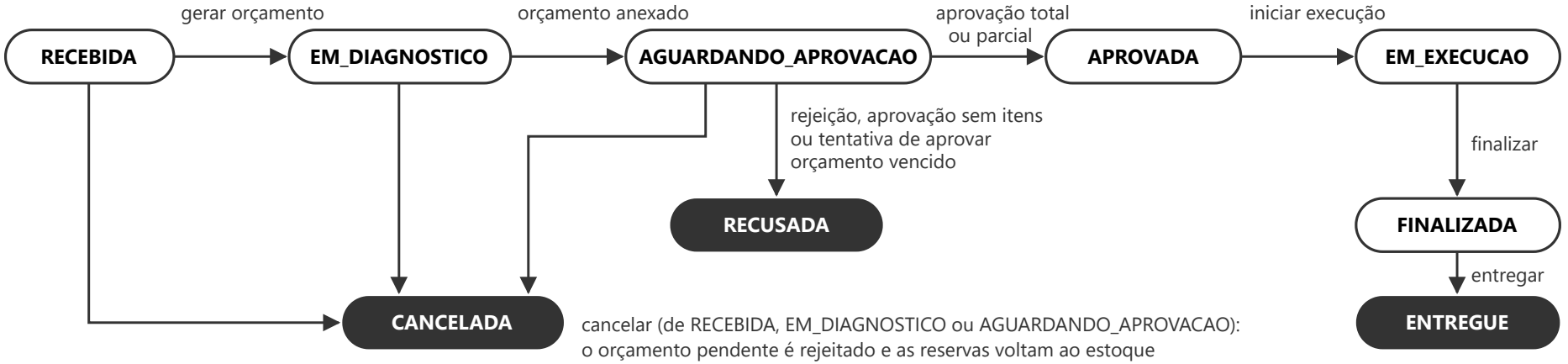
Relações

- **Clientes e Veículos (U) → Atendimento (D):** o núcleo define portas próprias (`VerificadorCliente`, `VerificadorVeiculo`, `ConsultorClientes`, `ConsultorVeiculos`) implementadas por adaptadores, que funcionam como camada anticorrupção.
- **Estoque (U) → Atendimento (D):** sem porta intermediária. Os casos de uso do orçamento e da OS usam o repositório de domínio `ProdutoEstoqueRepository` na mesma transação para reservar, baixar e devolver peças (cliente-fornecedor).
- **Catálogo (U) → Atendimento (D):** também sem porta, e o núcleo usa o modelo do catálogo como ele é (conformista). O `GerarOrcamentoUseCase` lê `ServicoRepository` na mesma transação e copia nome e valor para o `ItemServico`.
- **Atendimento (U) → E-mail (D):** eventos de domínio publicados depois do commit viram e-mail simulado. Não há WhatsApp nem PDF.
- **Autenticação (U) → Atendimento (D):** o JWT protege as rotas administrativas dos módulos.
- **Compartilhado:** só utilitário técnico (paginação, exceções, publicação de eventos e padrão de erros). Não é shared kernel: nenhum modelo de domínio é compartilhado entre os contextos.

Fonte: Miro (agregados) e código (módulos, portas e repositórios)

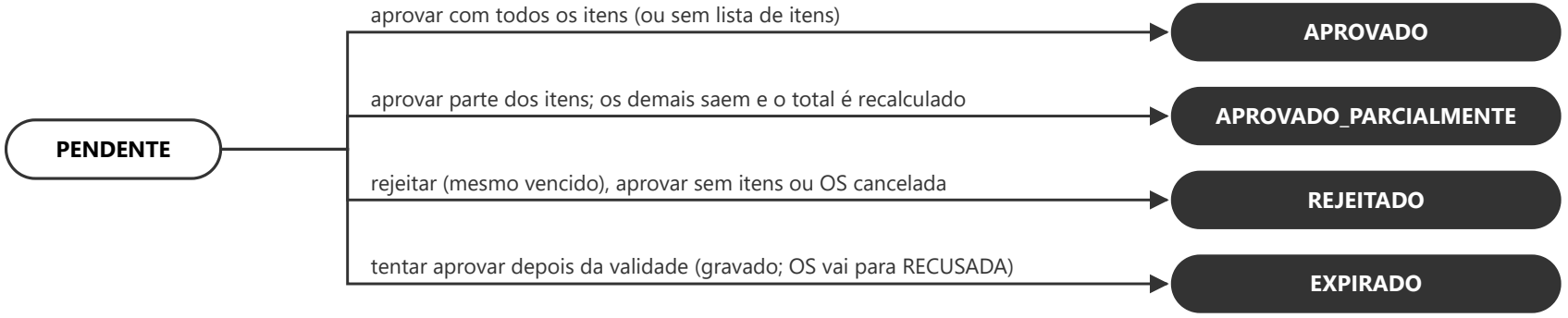
Máquinas de estado

Ordem de Serviço (`StatusOrdemServico`)



Estados em preto são finais. Cada mudança gera uma `TransicaoStatus` no histórico. Com essas datas dá para calcular o tempo médio de execução (intervalo entre `EM_EXECUCAO` e `FINALIZADA`), mas essa consulta ainda não foi implementada.

Orçamento (`StatusOrcamento`)



Todos os estados finais são em preto. A expiração só é gravada na tentativa de aprovação (`AprovarOrcamentoUseCase` chama `Orcamento.verificarExpiracao` e salva). Rejeitar um orçamento vencido resulta em `REJEITADO`, porque `Orcamento.rejeitar` não olha a validade. Na consulta do cliente a expiração aparece na resposta, mas a transação é somente leitura e não grava nada.

Fonte: código (`OrdemServico`, `Orcamento`, casos de uso de aprovação, rejeição e consulta)